

ASEN 6044

Homework #3

Bijan Jourabchi

Problem 1:

- a) Given the following prior information on independent state x , measurement bias z , and measurement likelihood $P(y | x, z)$

$$p(x) = \mathcal{U}[1, 11] \quad p(z) = \mathcal{G}(a, b) = \frac{1}{b^a \Gamma(a)} z^{a-1} e^{-z/b}$$

$$P(y | x, z) = \mathcal{U}[x, (x + z)^2]$$

with $a = b = 2$. We can derive the joint distribution as $P(x, z) = P(x)P(z)$ since x and z are independent. To determine the joint posterior PDF, $P(x, z | y_1, y_2, \dots, y_n)$ for a static state (not changing), we can use Bayes rule.

$$P(x, z | y_1, y_2, \dots, y_n) = \frac{P(x, z | y_1, y_2, \dots, y_{n-1})P(y_n | x, z)}{P(y_1, y_2, \dots, y_n)}$$

Thus, for the first measurement $y_1 = 91.56$ we have

$$P(x, z | y_1) = \frac{P(x, z)P(y_1 | x, z)}{P(y_1)}$$

We can derive similar expressions for subsequent measurements y_2 and y_3 ,

$$P(x, z | y_1, y_2) = \frac{P(x, z | y_1)P(y_2 | x, z)}{P(y_1, y_2)}$$

$$P(x, z | y_1, y_2, y_3) = \frac{P(x, z | y_1, y_2)P(y_3 | x, z)}{P(y_1, y_2, y_3)}$$

Coding this up using the given intervals for x and z , we can plot the joint posteriors seen in figures (1), (2), and (3). We see that after each subsequent measurement, the probability density changes slightly each time. This is due to each new measurement encoding more information on what the state could be.

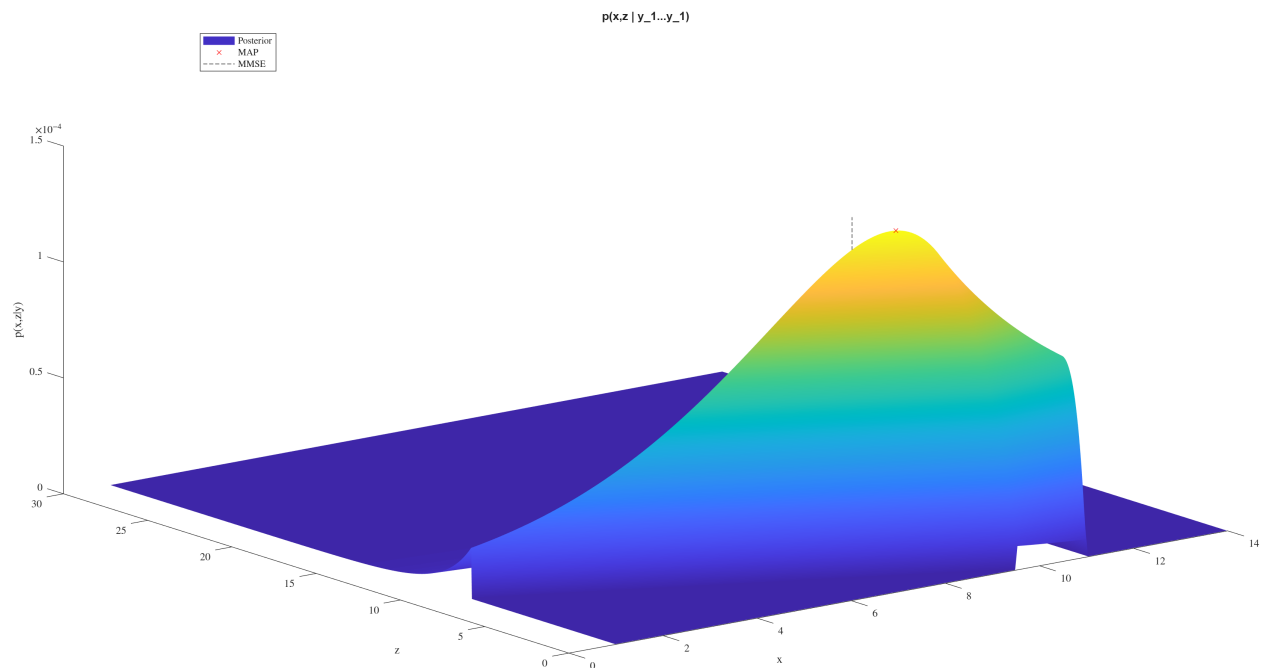


Figure 1: Joint Posterior $p(x, z | y_1)$

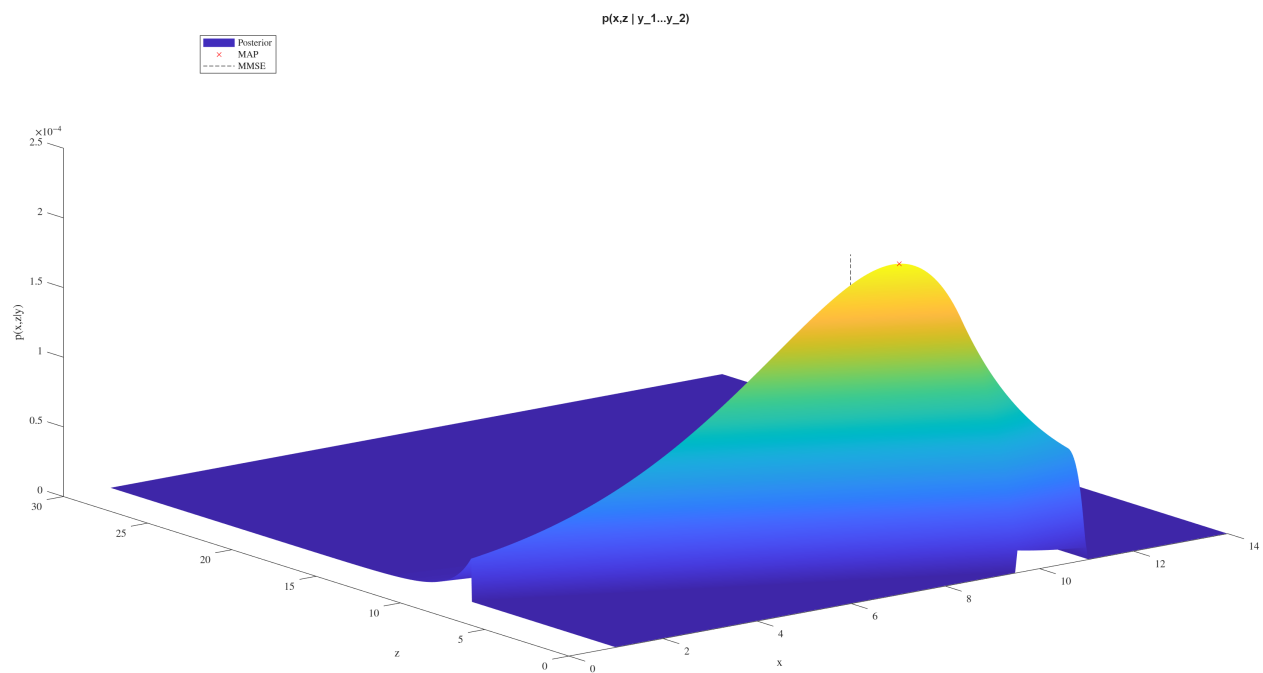
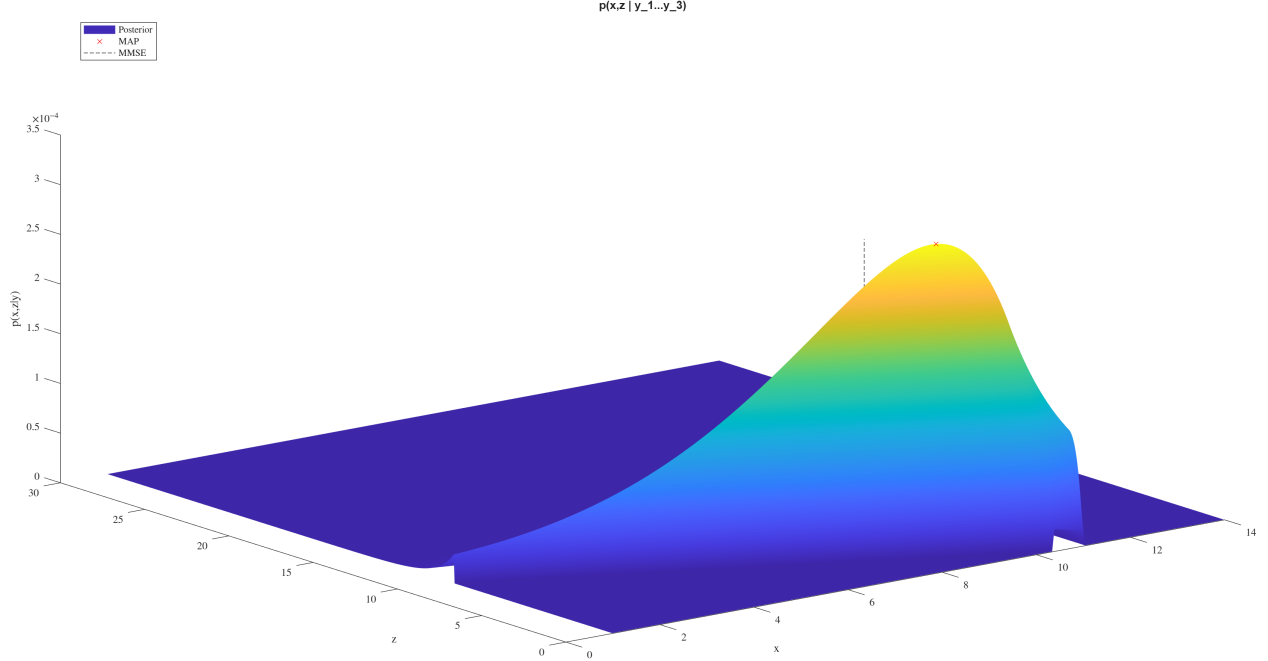


Figure 2: Joint Posterior $p(x, z | y_1, y_2)$

Figure 3: Joint Posterior $p(x, z | y_1, y_2, y_3)$

- b) Using the result from part a, we can easily determine the MMSE and MAP estimates of each joint posterior distribution. Table 1 summarizes these estimates for the three posterior distributions.

Table 1: MMSE and MAP estimates for the joint posterior distributions

Posterior	MMSE $\hat{z}$	MMSE $\hat{x}$	MAP $\hat{z}$	MAP $\hat{x}$
$p(x, z y_1)$	4.5064	7.6296	1.95	7.65
$p(x, z y_1, y_2)$	4.0683	7.4447	1.9	7.7
$p(x, z y_1, y_2, y_3)$	4.0755	7.7973	1.9	8.55

The value of the marginal data likelihoods $P(y_1), P(y_1, y_2), P(y_1, y_2, y_3)$ are the same as the normalization constants used when normalizing the joint posteriors. These values are $P(y_1) = 3.634 \times 10^{-3}$, $P(y_1, y_2) = 7.8 \times 10^{-3}$, and, $P(y_1, y_2, y_3) = 5.01 \times 10^{-3}$.

- c) Before performing Monte Carlo IS, we must derive an expression for the weights. The prior of $P(x, z | y_1, y_2, y_3)$ is just the joint posterior after receiving the second measurement, or $q(x, z; y_1, y_2, y_3) = P(x, z | y_1, y_2)$. The weights for IS are given as the ratio of the distribution we are trying to approximate, $P(x, z | y_1, y_2, y_3)$ to the proposal distribution. In this case we can simplify the expression for the importance weights.

$$w(X_s^i) = \frac{P(x_i, z_i | y_1, y_2, y_3)}{P(x_i, z_i | y_1, y_2)} \propto \frac{P(x_i, z_i | y_1, y_2)P(y_3 | x_i, z_i)}{P(x_i, z_i | y_1, y_2)} = P(y_3 | x_i, z_i)$$

Thus, the weight of a sampled point is proportional to the measurement likelihood of y_3 . Thus,

the particle weight for a sampled point x_s is

$$\tilde{w}(X_s^i) = \frac{w(X_s^i)}{\sum_{i=1}^{N_s} w(X_s^i)}$$

Performing IS with 500 samples, we get the result seen in figure (4). The MMSE estimate from the IS particles is the weighted sum of the particles. Computing this, we get a MMSE estimate of $\hat{x}_{MMSE,IS} = 4.21$ and $\hat{z}_{MMSE,IS} = 7.63$. Comparing this result from the MMSE value from Table 1, we see that the two are very similar. While the MMSE estimate from IS is slightly different than that of the exact grid based solution, importance sampling allows us to only sample the important areas of the state space. The loss of accuracy is traded off for the lighter computational load.

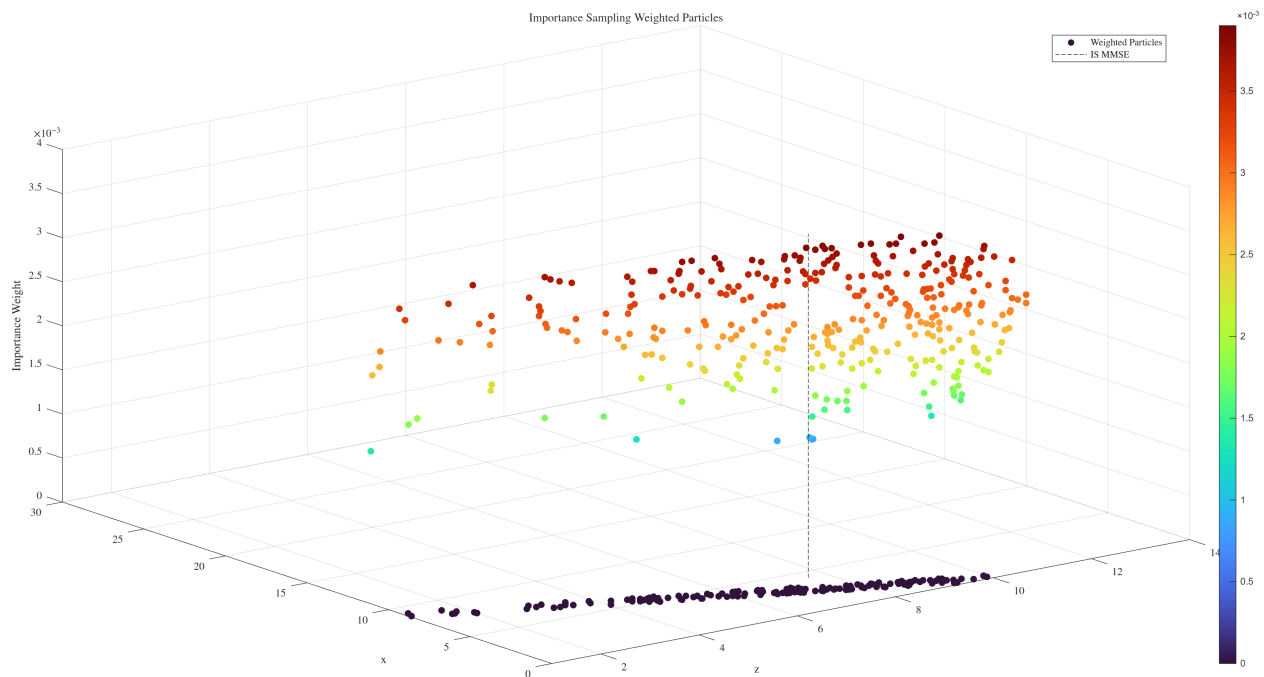


Figure 4: IS Estimate of $p(x, z \mid y_1, y_2, y_3)$

1.1 Code For Problem 1

```

1 clear; clc; close all
2
3 rng(500)
4 %% Define grid space + paramters
5
6 dx = 0.05;
7 x = 1:dx:14;
8 z = dx:dx:30;
9 a = 2;
10 b = 2;
11
12 Nx = length(x);
13 Nz = length(z);

```

```

14 %% Define prior grid dist
15
16 px = zeros(1,Nx);
17 px(x >= 1 & x <= 11) = 1/(11-1);
18
19 pz = (1/(b^a * gamma(a))) .* (z.^(a-1)) .* exp(-z./b);
20
21 P = zeros(Nx,Nz);
22
23 for i = 1:Nx
24     for j = 1:Nz
25         P(i,j) = px(i) * pz(j);
26     end
27 end
28
29 P = P / sum(P(:));
30
31 [X,Z] = meshgrid(z,x);
32
33 figure
34 surf(X,Z,P,'DisplayName','Prior')
35 xlabel('z')
36 ylabel('x')
37 zlabel('p(x,z)')
38 title('Joint Prior Distribution p(x,z)')
39 shading interp
40 legend('Location','best')
41
42 %% Posterior Bayes Update
43
44 y = [91.56, 70.43, 108.67];
45 Ny = 3;
46
47 L = likelihood(y(1),x,z);
48
49 figure
50 surf(X,Z,L,'DisplayName','Likelihood')
51 xlabel('z')
52 ylabel('x')
53 zlabel('p(y|x,z)')
54 title('Likelihood Dist p(y|x,z)')
55 shading interp
56 legend('Location','best')
57
58 for i = 1:Ny
59     Posterior = BayesUpdate(y(i),x,z,P, true,i);
60     P = Posterior;
61     MAP = getMAP(P,x,z);
62     MMSE = getMMSE(P,x,z);
63 end
64
65 %% IS
66
67 px = zeros(1,Nx);

```

```

68 px(x >= 1 & x <= 11) = 1/(11-1);
69
70 pz = (1/(b^a * gamma(a))) .* (z.^(a-1)) .* exp(-z./b);
71
72 P = zeros(Nx,Nz);
73
74 for i = 1:Nx
75     for j = 1:Nz
76         P(i,j) = px(i) * pz(j);
77     end
78 end
79
80 for i = 1:Ny-1
81     Posterior = BayesUpdate(y(i),x,z,P, false,i);
82     P = Posterior;
83     MAP = getMAP(P,x,z);
84     MMSE = getMMSE(P,x,z);
85 end
86
87 Ns = 500; % Sample Count
88
89 % Sample from prior
90 cdf_prior = cumsum(P(:));
91 random_values = rand(Ns, 1);
92 sample_indices = arrayfun(@(t) find(cdf_prior >= t, 1, 'first'),
93     random_values);
94 [row_indices, col_indices] = ind2sub(size(P), sample_indices);
95
96 xi = x(row_indices);
97 zi = z(col_indices);
98
99 w = zeros(1,500);
100
101 for i = 1:Ns
102     %w(i) =
103     likelihood(y(1),xi(i),zi(i))*likelihood(y(2),xi(i),zi(i))*likelihood(y(3),xi(i),zi(i))
104     w(i) = likelihood(y(3),xi(i),zi(i));
105 end
106
107 w = w./sum(w); % Normalize
108
109 % MMSE estimate
110 IS_MMSE = [0;0];
111 for i = 1:Ns
112     IS_MMSE = IS_MMSE + w(i)*[zi(i);xi(i)];
113 end
114
115 figure; hold on
116 scatter3(xi, zi, w, 40, w, 'filled', 'DisplayName', 'Weighted Particles')
117 plot3([IS_MMSE(1) IS_MMSE(1)], [IS_MMSE(2) IS_MMSE(2)], [0 max(w)], 'k--',
118     'DisplayName', 'IS MMSE')
119 xlim([1 14])

```

```

119 ylim([0 30])
120 xlabel('z')
121 ylabel('x')
122 zlabel('Importance Weight')
123 title('Importance Sampling Weighted Particles')
124
125 colorbar
126 colormap turbo
127 grid on
128 legend('Location','best')
129 view(3)
130 exportgraphics(gcf, "IS.png", 'Resolution', 200);
131
132 %% Helper Functions
133
134 function [L] = likelihood(y,x,z)
135
136 Nx = length(x);
137 Nz = length(z);
138 L = zeros(Nx,Nz);
139
140 for i = 1:Nx
141     for j = 1:Nz
142         % Out of bounds
143         if y < x(i) || y > (x(i)+z(j))^2
144             L(i,j) = 0;
145             continue
146         end
147
148         L(i,j) = 1/((x(i)+z(j))^2-x(i));
149     end
150 end
151
152
153 end
154
155 function [Posterior] = BayesUpdate(y,x,z,Prior, PLOT,i)
156
157 L = likelihood(y,x,z);
158
159 Posterior = Prior.*L;
160 fprintf("Value of Marginal P(y_1:N) = %d for meas 1:%i\n",sum(Posterior(:)),i)
161 Posterior = Posterior./(sum(Posterior(:)));
162
163
164
165 if PLOT
166     MAP = getMAP(Posterior,x,z);
167     MMSE = getMMSE(Posterior,x,z);
168     [X,Z] = meshgrid(z,x);
169     figure
170     hold on
171     surf(Z,X,Posterior,'DisplayName','Posterior')

```

```

172     scatter3(MAP(1),MAP(2),MAP(3),'rx','DisplayName','MAP')
173     plot3([MMSE(1) MMSE(1)],[MMSE(2) MMSE(2)],[0
        MAP(3)],'k--','DisplayName','MMSE')
174     xlabel('x')
175     ylabel('z')
176     zlabel('p(x,z|y)')
177     title(sprintf('p(x,z | y_1...y_%d)',i))
178     shading interp
179     legend('Location','best')
180     view(3)
181     exportgraphics(gcf, sprintf('Measurment_%d',i) + ".png",
        'Resolution', 200);
182 end
183 end
184
185 function [MAP] = getMAP(P,x,z)
186
187     [peak,idx] = max(P(:));
188     [row, col] = ind2sub(size(P), idx);
189     [X,Z] = meshgrid(z,x);
190     map_z = X(row,col);
191     map_x = Z(row,col);
192
193     MAP = [map_z;map_x;peak];
194
195 end
196
197 function [MMSE] = getMMSE(P,x,z)
198 [X,Z] = meshgrid(z,x);
199
200 z_mmse = sum(sum(P .* X));
201 x_mmse = sum(sum(P .* Z));
202
203 [~, ix] = min(abs(x - x_mmse));
204 [~, iz] = min(abs(z - z_mmse));
205
206 MMSE = [z_mmse; x_mmse; P(ix,iz)];
207
208 end

```

Problem 2:

- a) Given the state index transition probabilities, we can compile all of the transition information into a matrix $T \in \mathbb{R}^{N_x \times N_x}$ with

$$T_{ij} = P(\zeta_{k+1} = i \mid \zeta_k = j), \quad \rho_{k+1} = T\rho_k.$$

Therefore, column j corresponds to the current state $\zeta_k = j$, while row i corresponds to the propagated state $\zeta_{k+1} = i$.

$$T = \begin{bmatrix} 0.1 & 0.2 & 0 & 0 & \cdots & 0 & 0 \\ 0.9 & 0.3 & 0.2 & 0 & \cdots & 0 & 0 \\ 0 & 0.5 & 0.3 & 0.2 & \ddots & \vdots & \vdots \\ 0 & 0 & 0.5 & 0.3 & \ddots & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & 0.2 & 0 \\ 0 & 0 & \cdots & 0 & 0.5 & 0.3 & 0.9 \\ 0 & 0 & \cdots & 0 & 0 & 0.5 & 0.1 \end{bmatrix}.$$

A stationary distribution ρ_∞^- such that $\rho_\infty^- = T\rho_\infty^-$ will exist if the square matrix T has an eigenvalue $\lambda = 1$. In such case, the matrix eigenvalue equation becomes $T\vec{v} = \vec{v}$ where $\vec{v}$ is the eigenvector corresponding to $\lambda = 1$. By definition of a stationary distribution, the eigenvector corresponding to $\lambda = 1$ is our stationary distribution. Since T has an eigenvalue $\lambda = 1$, there exists a stationary distribution ρ_∞^- .

b) To derive a general expression for ρ_k^- in terms of ρ_0^- , we start with the following

$$\begin{aligned} \rho_1^- &= T\rho_0^- \\ \rho_2^- &= T\rho_1^- = TT\rho_0^- = T^2\rho_0^- \\ \rho_3^- &= T\rho_2^- = T^3\rho_0^- \end{aligned}$$

This can be generalized so that at time step k we have

$$\rho_k^- = T^k \rho_0^-$$

This distribution is plotted for $k = 10, 50, 180, 300, 600$ in figures 5a - 5e. ρ_∞^- is shown in figure 5f. We see that ρ_{600}^- is identical to the stationary distribution, however figure 5d shows us that after approximately 300 timesteps, ρ^- gets very close to the stationary distribution.

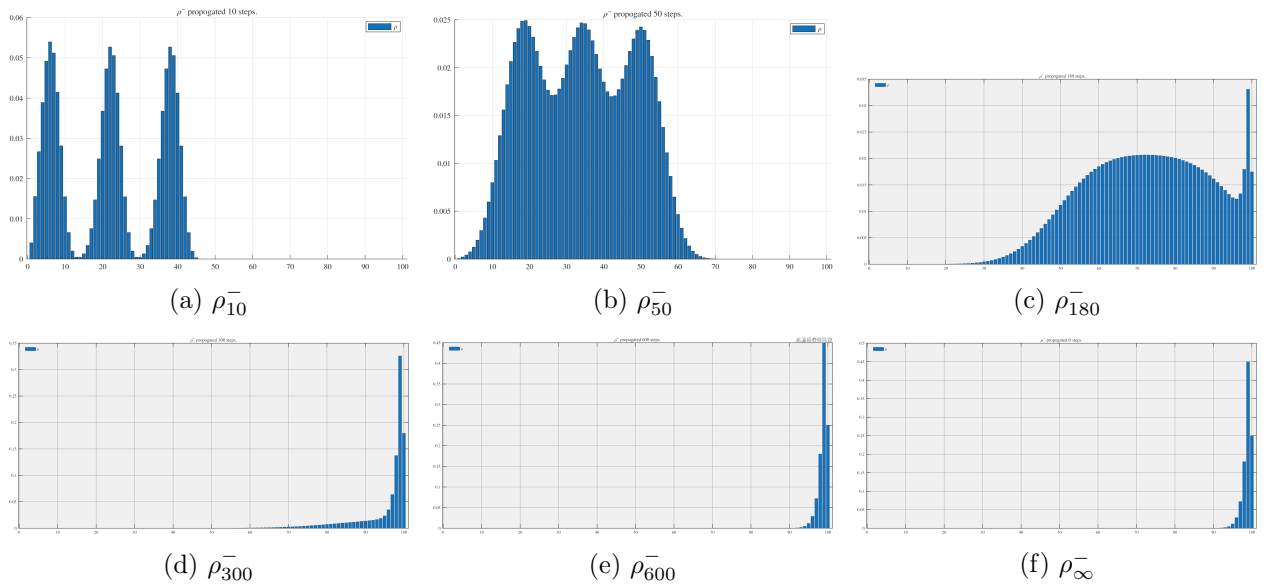


Figure 5: Propagated distributions ρ_k^- for selected k .

- c) To implement a recursive Bayes filter to compute a posterior ρ_k^+ , we can perform a Bayes update on our prior ρ_k^- . The full recursive Bayes filter for this problem is

$$\begin{aligned}\rho_k^- &= T\rho_{k-1}^+ \\ \rho_k^+ &\propto \mathcal{L}(z_k; \zeta_k) \rho_k^- \\ \mathcal{L}(z_k; \zeta_k) &= P(z_k = i \mid \zeta_k = j) = \mathcal{U}_{i|j}[a(r), b(r)]\end{aligned}$$

The results at the selected timesteps are summarized in figure 6 and table 2. Compared to the truth ζ values we see that the exact Bayes filter MMSE and MAP estimates are both very close to the ground truth values.

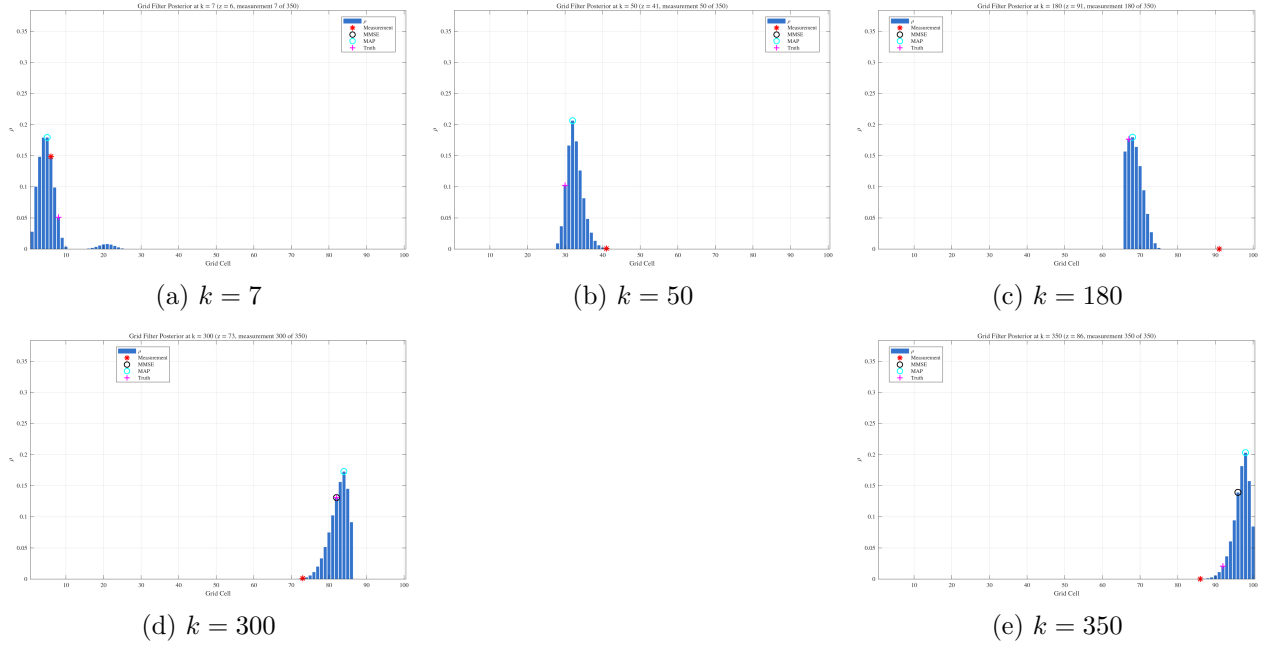


Figure 6: Grid-filter posterior snapshots at selected timesteps.

Table 2: Grid-filter MMSE and MAP estimates at selected timesteps

Timestep k	MMSE Cell Index	MAP Cell Index	Ground Truth
7	5	4	7
50	32	31	30
180	67	67	66
300	82	84	82
350	96	97	93

- d) The bootstrap particle filter sets the proposal distribution as the transition pdf $P(\zeta_{k+1} = i \mid \zeta_k + j)$ and resamples after each measurement weight update. The choice of importance distribution allows the weight update to simplify to observation likelihood which makes implementation very easy. Figure 7 shows the approximate posterior at the specified time steps using the weighted sum of particles to approximate the distribution shape. The MMSE and

MAP estimates are summarized in table 3. The estimates are very similar to the exact Bayes grid filter indicating that the bootstrap PF does a very good job at approximating the exact distribution. The MMSE and MAP estimates across each timestep for the grid and PF are summarized in figure 8.

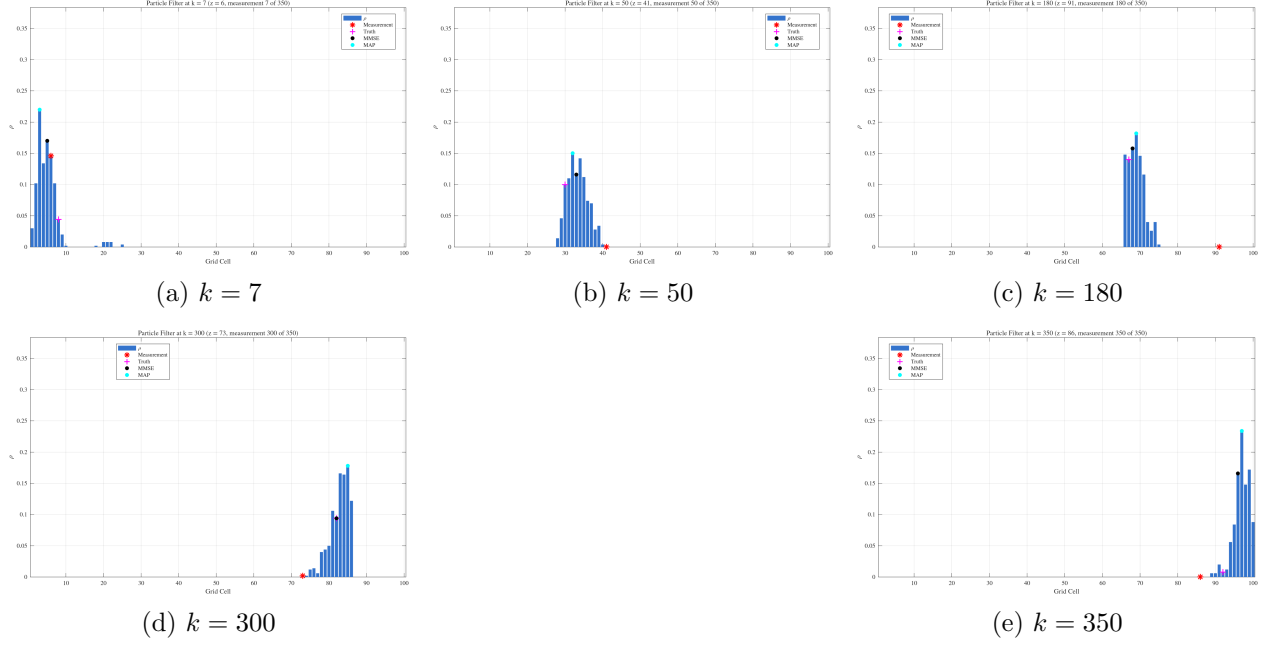
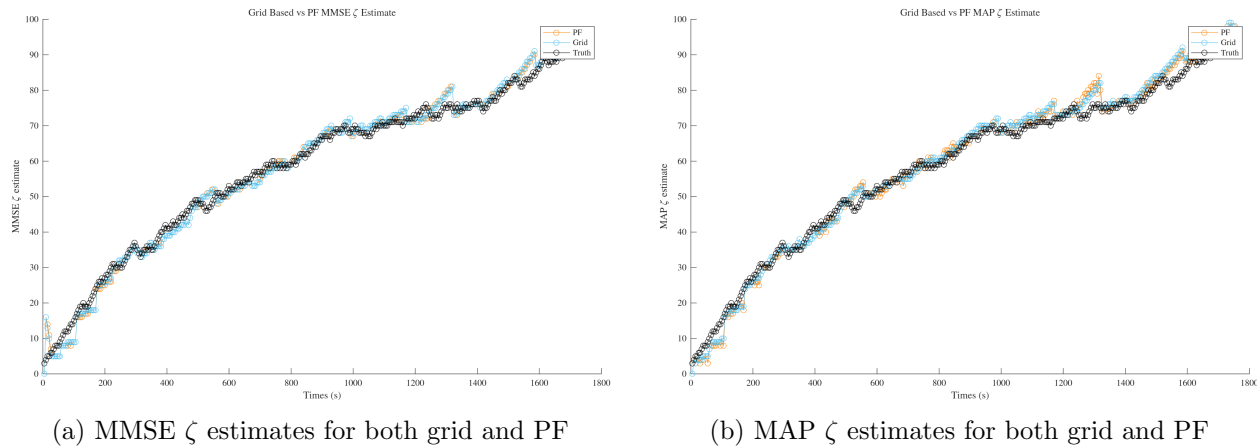


Figure 7: PF posterior snapshots at selected timesteps.

Table 3: PF MMSE and MAP estimates at selected timesteps

Timestep k	MMSE Cell Index	MAP Cell Index	Ground Truth
7	5	4	7
50	32	31	30
180	66	67	66
300	81	82	82
350	96	98	93

Figure 8: Grid and PF ζ estimates

2.1 Code For Problem 2

```

1 clear; clc; close all
2 load("asen6044hw3_problem2.mat")
3
4
5 set(groot, 'defaultTextInterpreter', 'latex')
6 set(groot, 'defaultAxesTickLabelInterpreter', 'latex')
7 set(groot, 'defaultLegendInterpreter', 'latex')
8
9 %% Declare Constants
10
11 rng(400)
12 r = 25;
13 L = 1000;
14 dx = 10;
15 dt = 5;
16 Nx = 100;
17
18 %% Part a) Get T and rho_inf
19
20 T = getT(Nx);
21
22 [V,D] = eig(T);
23
24 idx = find(abs(diag(D) - 1) < 1e-10);
25 rhoinf = V(:, idx);
26 % corresponding eigenvector
27 rhoinf(rhoinf < 0) = 0;
28 rhoinf = rhoinf./sum(rhoinf); % Normalize
29
30 %% Part b
31
32 rho0 = zeros(Nx,1);
33 rho0(3) = 1/3;
34 rho0(19) = 1/3;
35 rho0(35) = 1/3;

```

```

36
37 % prop_rho_kSteps(rho0,T,10, true);
38 % prop_rho_kSteps(rho0,T,50, true);
39 % prop_rho_kSteps(rho0,T,180, true);
40 % prop_rho_kSteps(rho0,T,300, true);
41 % prop_rho_kSteps(rho0,T,600, true);
42 % prop_rho_kSteps(rho0,T,0, true);
43
44 %% Part c
45
46
47 saveGridGif = true;
48 gifFile = "grid_filter.gif";
49 saveFrames = [7 50 180 300 350];
50 gridGif(zMeasHist,rho0,T,r,zetaHist, 'SaveGif', saveGridGif, 'GifFile',
    gifFile, 'SaveFrames', saveFrames);
51
52 %% Part d
53
54 % Initialize PF
55 particles = cell(1,length(zMeasHist));
56
57 Ns = 500;
58
59 InitCells = [3 19 35];
60
61 random_indices = randi([1, length(InitCells)], Ns, 1);
62
63 particles{1}.x = InitCells(random_indices);
64 particles{1}.w = 1/Ns * ones(1,Ns);
65 particles{1}.MMSE = 0;
66 particles{1}.MAP = 0;
67 particles{1}.Nx = Nx;
68
69 for i = 2:length(zMeasHist)
70     particles{i} = PF(particles{i-1},T,zMeasHist(i),Ns);
71 end
72
73
74 PFgif(zMeasHist,particles{1},T,zetaHist,rho0,'SaveGif',true,'GifFile',"pf_filter.gif",
75
76 %% MMSE and MAP comparison
77 Nm = length(zMeasHist);
78 rho_prior = rho0;
79 MMSE_Grid = zeros(1,Nm);
80 MAP_Grid = zeros(1,Nm);
81 MMSE_PF = zeros(1,Nm);
82 MAP_PF = zeros(1,Nm);
83 Like = meas_like(r,Nx);
84 Nm = length(zMeasHist);
85 for k = 2:Nm
86     rho_minus = T * rho_prior;
87     rho_plot = Like(zMeasHist(k), :) .* rho_minus;
88     rho_plot = rho_plot ./ sum(rho_plot);

```

```

89     MMSE_Grid(k) = getMMSE(rho_plot);
90     MAP_Grid(k) = getMAP(rho_plot);
91     MMSE_PF(k) = particles{k}.MMSE;
92     MAP_PF(k) = particles{k}.MAP;
93     rho_prior = rho_plot;
94 end
95
96 figure; hold on
97 tvec = dt:dt:dt*Nm;
98 plot(tvec,MMSE_PF,"-o","Color",[1, 0.5, 0])
99 plot(tvec,MMSE_Grid,"-o", "Color",[0.3010, 0.7450, 0.9330])
100 plot(tvec,zetaHist,"k-o")
101 xlabel("Times (s)")
102 ylabel("MMSE $\zeta$ estimate")
103 title("Grid Based vs PF MMSE $\zeta$ Estimate")
104 legend("PF","Grid","Truth")
105 exportgraphics(gcf, "mmse_estimate_comparison.png", "Resolution", 200);
106
107 figure; hold on
108 tvec = dt:dt:dt*Nm;
109 plot(tvec,MMSE_PF-zetaHist,"-o","Color",[1, 0.5, 0])
110 plot(tvec,MMSE_Grid-zetaHist,"-o", "Color",[0.3010, 0.7450, 0.9330])
111 xlabel("Times (s)")
112 ylabel("MMSE $\zeta$ estimate error")
113 title("Grid Based vs PF MMSE $\zeta$ Estimate Error")
114 legend("PF","Grid")
115 exportgraphics(gcf, "mmse_error_comparison.png", "Resolution", 200);
116
117 figure; hold on
118 tvec = dt:dt:dt*Nm;
119 plot(tvec,MAP_PF,"-o","Color",[1, 0.5, 0])
120 plot(tvec,MAP_Grid,"-o", "Color",[0.3010, 0.7450, 0.9330])
121 plot(tvec,zetaHist,"k-o")
122 xlabel("Times (s)")
123 ylabel("MAP $\zeta$ estimate")
124 title("Grid Based vs PF MAP $\zeta$ Estimate")
125 legend("PF","Grid","Truth")
126 exportgraphics(gcf, "map_estimate_comparison.png", "Resolution", 200);
127
128 figure; hold on
129 tvec = dt:dt:dt*Nm;
130 plot(tvec,MAP_PF-zetaHist,"-o","Color",[1, 0.5, 0])
131 plot(tvec,MAP_Grid-zetaHist,"-o", "Color",[0.3010, 0.7450, 0.9330])
132 xlabel("Times (s)")
133 ylabel("MAP $\zeta$ estimate error")
134 title("Grid Based vs PF MAP $\zeta$ Estimate Error")
135 legend("PF","Grid")
136 exportgraphics(gcf, "map_error_comparison.png", "Resolution", 200);
137
138 %% Helper Functions
139
140 function [particles] = PF(particles,T,z,Ns)
141
142 r = 25;

```

```

143 Nx = size(T,1);
144
145 Like = meas_like(r,Nx);
146
147 x = particles.x;
148 w = particles.w;
149
150 xp = zeros(1,Ns);
151 wp = zeros(1,Ns);
152
153 for i = 1:Ns
154     % Bootstrap PF proposal: sample  $x_k^i \sim p(x_k | x_{k-1}^i)$ 
155     xp(i) = sampleState(T(:,x(i)));
156
157     % SIS weight update:  $w_k^i$  proportional to  $w_{k-1}^i * p(z_k | x_k^i)$ 
158
159     wp(i) = Like(z,xp(i))/Ns;
160 end
161
162 wp = wp./sum(wp);
163
164 particles.MMSE = floor(sum(xp .* wp));
165 particles.MAP = getParticleMAP(xp,wp,Nx);
166
167 % RESAMPLE
168 for i = 1:Ns
169     p_idx = sampleState(wp);
170     particles.x(i) = xp(p_idx);
171 end
172
173 particles.w = 1/Ns * ones(1,Ns);
174 particles.Nx = Nx;
175
176 end
177
178 function [] = plotPF(p, Ns, k, rho0,varargin)
179
180     z = NaN;
181     zeta = NaN;
182     fig = [];
183
184     if mod(numel(varargin),2) ~= 0
185         error('Optional inputs must be provided as name/value pairs.');

```

```

197         case "figure"
198             fig = value;
199         otherwise
200             error("Unknown option '%s'.", varargin{i});
201     end
202 end
203
204 if isempty(fig)
205     figure;
206 else
207     figure(fig);
208 end
209
210 clf;
211 hold on;
212 grid on;
213 box on;
214
215 if isfield(p, 'Nx')
216     Nx = p.Nx;
217 else
218     Nx = max(p.x(:));
219 end
220
221 rho = particlesToRho(p, Nx);
222 MMSE = getMMSE(rho);
223 MAP = getMAP(rho);
224
225 bar(1:Nx, rho, 'FaceColor', [0.2 0.45 0.8], 'EdgeColor', 'none', ...
226     'DisplayName', '$\rho$');
227
228
229 if ~isnan(z)
230     plot(z, rho(z), 'r*', 'MarkerSize', 10, 'LineWidth', 1.5, ...
231         'DisplayName', 'Measurement');
232 end
233
234 if ~isnan(zeta)
235     plot(zeta, rho(zeta), 'm+', 'MarkerSize', 10, 'LineWidth', 1.5, ...
236         'DisplayName', 'Truth');
237 end
238
239 plot(MMSE, rho(MMSE), 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 6, ...
240     'DisplayName', 'MMSE');
241 plot(MAP, rho(MAP), 'co', 'MarkerFaceColor', 'c', 'MarkerSize', 6, ...
242     'DisplayName', 'MAP');
243
244 xlim([0.5, Nx + 0.5])
245 ylim([0, max([rho0(:); rho(:)]) * 1.15 + eps]);
246 xlabel('Grid Cell')
247 ylabel('$\rho$')
248 title("PF estimated distribution at timestep k = " + num2str(k))

```



```

249     legend('Location', 'best')
250
251     hold off;
252
253 end
254
255 function [] = PFgif(z,particles0,T,zeta,rho0,varargin)
256 Ns = numel(particles0.x);
257
258 saveGif = false;
259 gifFile = "pf_filter.gif";
260 delayTime = 0.35;
261 saveFrames = [];
262
263 if mod(numel(varargin),2) ~= 0
264     error('Optional inputs must be provided as name/value pairs.');
```

```

265 end
266
267 for i = 1:2:numel(varargin)
268     name = lower(string(varargin{i}));
269     value = varargin{i+1};
270
271     switch name
272     case "savegif"
273         saveGif = logical(value);
274     case "giffile"
275         gifFile = string(value);
276     case "delaytime"
277         delayTime = value;
278     case "saveframes"
279         saveFrames = value(:)';
280     otherwise
281         error("Unknown option '%s'.", varargin{i});
282     end
283 end
284
285 particles = particles0;
286 numMeas = numel(z);
287 measIdx = find(~isnan(z(:)));
288
289 fig = figure;
290 for k = 1:numMeas
291     if k > 1
292         particles = PF(particles,T,z(k),Ns);
293     end
294
295     plotPF(particles, Ns, k-1, rho0,...
296         'Measurement', z(k), ...
297         'Truth', zeta(k), ...
298         'Figure', fig);
299
300     if k == 1
301         title('Initial Particle Prior at k = 0');
302     elseif isnan(z(k))

```

```

303     title(sprintf('Particle Prior at k = %d (no measurement)', k-1));
304 else
305     title(sprintf('Particle Filter at k = %d (z = %d, measurement %d
    of %d)', ...
306         k-1, z(k), find(measIdx == k, 1), numel(measIdx)));
307 end
308
309 drawnow;
310
311 if saveGif
312     frame = getframe(fig);
313     [imind, cm] = rgb2ind(frame2im(frame), 256);
314     if k == 1
315         imwrite(imind, cm, char(gifFile), 'gif', 'LoopCount', inf, ...
316             'DelayTime', delayTime);
317     else
318         imwrite(imind, cm, char(gifFile), 'gif', 'WriteMode',
319             'append', ...
320             'DelayTime', delayTime);
321     end
322 end
323
324 if any(saveFrames == k-1)
325     saveGifFrame(fig, gifFile, k-1);
326 end
327 end
328
329 function [idx] = sampleState(p)
330
331     c = cumsum(p);
332     u = rand;
333     idx = find(u <= c, 1, 'first');
334
335 end
336
337 function [MAP] = getParticleMAP(x,w,Nx)
338     mass = accumarray(x(:), w(:), [Nx 1]);
339     MAP = find(mass == max(mass), 1, 'first');
340 end
341
342 function [rho] = particlesToRho(p, Nx)
343 x = p.x(:);
344 w = p.w(:);
345 rho = accumarray(x, w, [Nx 1]); % Sum probability mass at each grid cell.
346 rho = rho ./ sum(rho);
347 end
348
349 function [] = saveGifFrame(fig, gifFile, k)
350     [folder, name, ~] = fileparts(char(gifFile));
351     pngFile = fullfile(folder, sprintf('%s_frame_%03d.png', name, k));
352     exportgraphics(fig, pngFile, 'Resolution', 200);
353 end
354

```

```

355 function [T] = getT(Nx)
356
357 T = zeros(Nx);
358 for i = 1:Nx
359     for j = 1:Nx
360         if i == j && i > 1 && i <= Nx-1
361             T(i,j) = 0.3;
362         elseif i == j+1 && i > 2 && j >= 2
363             T(i,j) = 0.5;
364         elseif i == j-1 && i <= Nx-2 && j <= Nx-1
365             T(i,j) = 0.2;
366         elseif i == 1 && j == 1
367             T(i,j) = 0.1;
368         elseif i == Nx && j == Nx
369             T(i,j) = 0.1;
370         elseif i == 2 && j == 1
371             T(i,j) = 0.9;
372         elseif i == Nx-1 && j == Nx
373             T(i,j) = 0.9;
374         end
375     end
376 end
377 end
378
379 %% Plot rho dist as bar plot
380 function [] = plotRho(rho,k,z)
381
382 if nargin < 3
383     z = NaN;
384 end
385
386 figure;
387 hold on;
388 grid on;
389
390 bar(rho, 'DisplayName', '$\rho$')
391 if ~isnan(z)
392     plot(z,rho(z)+0.03, 'r*', 'MarkerSize', 12, 'LineWidth', 2, ...
393         'DisplayName', 'Measurement')
394 end
395 title("$\rho^-$ propagated " + num2str(k) + " steps.")
396 legend('Location', 'best')
397 exportgraphics(gcf, sprintf('rho_propagated_%03d.png', k), 'Resolution',
398     200);
399 end
400
401 %% Propagate + Plot
402 function [rho_minus] = prop_rho_kSteps(rho0,T,k, PLOT)
403
404 rho_minus = T^k * rho0;
405
406 if PLOT
407     plotRho(rho_minus,k)

```

```

408 end
409
410 end
411
412 %% Measurment Likelihood Matrix
413 function [L] = meas_like(r,Nx)
414 L = zeros(Nx);
415 for j = 1:Nx
416
417     a = max([1, j-r]);
418     b = min([Nx, j+r]);
419
420     U = 1/(b-a+1);
421
422     for i = 1:Nx
423         if i >= a && i <= b
424             L(i,j) = U;
425         end
426     end
427 end
428
429 end
430
431 %% Grid Bayes Filter
432 function [rho_plus] = prop_rho_kSteps_wMeas(z,rho0,T,k, r,Nx,PLOT)
433 Like = meas_like(r,Nx); % Const for each timestep
434 rho_prior = rho0;
435 for j = 1:k-1
436     rho_minus = T*rho_prior;
437     rho_plus = Like(z(j+1),:)'.*rho_minus;
438     rho_plus = rho_plus./sum(rho_plus);
439     rho_prior = rho_plus;
440 end
441 if PLOT
442     plotRho(rho_plus,k,z(k))
443 end
444 end
445
446 %% Full Grid Bayes Filter Gif
447 function [] = gridGif(z,rho0,T,r,zeta,varargin)
448 Nx = size(T,1);
449
450 saveGif = false;
451 gifFile = "grid_filter.gif";
452 delayTime = 0.35;
453 saveFrames = [];
454
455 if mod(numel(varargin),2) ~= 0
456     error('Optional inputs must be provided as name/value pairs.');
```

```

462
463     switch name
464         case "savegif"
465             saveGif = logical(value);
466         case "giffile"
467             gifFile = string(value);
468         case "delaytime"
469             delayTime = value;
470         case "saveframes"
471             saveFrames = value(:)';
472         otherwise
473             error("Unknown option '%s'.", varargin{i});
474     end
475 end
476
477 Like = meas_like(r,Nx);
478 rho_prior = rho0(:);
479 numMeas = numel(z);
480 measIdx = find(~isnan(z(:)));
481
482 fig = figure;
483 for k = 1:numMeas
484     clf(fig);
485
486     if k == 1 || isnan(z(k))
487         rho_plot = rho_prior;
488         bar(1:Nx, rho_plot, 'FaceColor', [0.2 0.45 0.8], 'EdgeColor',
489             'none', ...
490             'DisplayName', '$\rho$');
491         if k == 1
492             title('Initial Prior at k = 0');
493         else
494             title(sprintf('Prior at k = %d (no measurement)', k-1));
495         end
496         legend('Location', 'best');
497     else
498         rho_minus = T * rho_prior;
499         rho_plot = Like(z(k), :)'.* rho_minus;
500         rho_plot = rho_plot ./ sum(rho_plot);
501         MMSE = getMMSE(rho_plot);
502         MAP = getMAP(rho_plot);
503         bar(1:Nx, rho_plot, 'FaceColor', [0.2 0.45 0.8], 'EdgeColor',
504             'none', ...
505             'DisplayName', '$\rho$');
506         hold on;
507         plot(z(k), rho_plot(z(k)), 'r*', 'MarkerSize', 10, 'LineWidth',
508             1.5, ...
509             'DisplayName', 'Measurement');
510         plot(MMSE, rho_plot(MMSE), 'ko', 'MarkerSize', 10, 'LineWidth',
511             1.5, ...
512             'DisplayName', 'MMSE');
513         plot(MAP, rho_plot(MAP), 'co', 'MarkerSize', 10, 'LineWidth',
514             1.5, ...
515             'DisplayName', 'MAP');

```

```

511     plot(zeta(k),rho_plot(zeta(k)), 'm+', 'MarkerSize', 10,
512           'LineWidth', 1.5, ...
513           'DisplayName', 'Truth');
514     hold off;
515     legend('Location', 'best');
516     title(sprintf('Grid Filter Posterior at k = %d (z = %d,
517                   measurement %d of %d)', ...
518                   k-1, z(k), find(measIdx == k, 1), numel(measIdx)));
519     rho_prior = rho_plot;
520 end
521
522 grid on;
523 xlim([0.5, Nx + 0.5]);
524 ylim([0, max([rho0(:); rho_plot(:)]) * 1.15 + eps]);
525 xlabel('Grid Cell');
526 ylabel('$\rho$');
527
528 drawnow;
529
530 if saveGif
531     frame = getframe(fig);
532     [imind, cm] = rgb2ind(frame2im(frame), 256);
533     if k == 1
534         imwrite(imind, cm, char(gifFile), 'gif', 'LoopCount', inf, ...
535               'DelayTime', delayTime);
536     else
537         imwrite(imind, cm, char(gifFile), 'gif', 'WriteMode',
538               'append', ...
539               'DelayTime', delayTime);
540     end
541 end
542
543 if any(saveFrames == k-1)
544     saveGifFrame(fig, gifFile, k-1);
545 end
546 end
547 end
548
549 function [MMSE] = getMMSE(rho)
550
551 MMSE = 0;
552 for j = 1:length(rho)
553     MMSE = MMSE + rho(j)*j;
554 end
555 MMSE = floor(MMSE);
556 end
557
558 function [MAP] = getMAP(rho)
559 MAP = find(rho == max(rho));
560 end

```